

無限井字遊戲專題實作： 動態配對與即時同步雲端架構

林柏翰
資工二甲
國立澎湖科技大學

Abstract—傳統井字棋受限於棋盤空間，極易產生平局，缺乏長期策略性。本專案開發了一款基於 Web 技術的「無限井字遊戲 (Infinite Tic-Tac-Toe)」，引入動態印記消除機制 (FIFO) 解決平局痛點。系統底層採用 Firebase Realtime Database 結合 WebSocket 達成事件驅動的即時狀態同步，並實作基於原子化交易 (runTransaction) 的動態配對機制，徹底分離大廳等待與遊戲本體邏輯。本報告將探討該遊戲的核心演算法、雲端同步架構以及防範 Race Condition 的實作細節。

Index Terms—Tic-Tac-Toe, Firebase Realtime Database, WebSocket, Matchmaking, Queue (FIFO)

I. INTRODUCTION

井字棋作為經典的益智遊戲，其 3×3 的空間限制使得熟練玩家極易進入「必和局」的僵局。為了解決此缺乏變數的遊戲機制，本研究提出「無限規則」構想：限制每位玩家在場上最多僅能保留 3 個印記。透過陣列的先進先出 (First-In-First-Out, FIFO) 輪轉機制，棋盤狀態將隨著回合推進而不斷變化，迫使玩家進行更深層的佈局考量。

本專案除了創新遊戲規則外，在網路架構上亦採用了雲端無伺服器 (Serverless) 設計，利用 Firebase 實現零延遲的多人連線對戰，並探討了在分散式系統中如何妥善處理玩家斷線與併發配對的問題。

II. 遊戲核心機制設計

A. 三印記限制與 FIFO 消除

遊戲的核心邏輯圍繞著「動態存留」展開。每位玩家 (藍方 O 或紅方 X) 的印記座標皆被儲存於一個獨立的 Queue 資料結構中。當玩家嘗試放置第 4 個印記時，系統會觸發 `shift()` 操作，將最舊的座標移出陣列並從棋盤上抹除。此機制確保了戰局永遠處於變化中，從根本上消除了絕對平手的可能性。

B. 連線獲勝判定之時序控制

在無限井字遊戲的演算法設計中，獲勝判定 (Win Condition Check) 的時序至關重要。當玩家放置第 4 個印記的瞬間，系統必須「優先執行」空間掃描判定 (橫、直、斜共 8 種連線可能)。若滿足三連線條件，系統會立即將狀態寫入 `status: 'finished'` 並宣告獲勝；若未達成連線，系統才會執行前述的 FIFO 消除邏輯。此時序確保了玩家的「致勝一手」不會因為舊印記的消失而失效。從圖 1 的 UI 設計中亦可看出，系統除了核心棋盤外，還整合了當前回合提示、時間計算與悔棋次數統計，以提供完整的遊戲體驗。

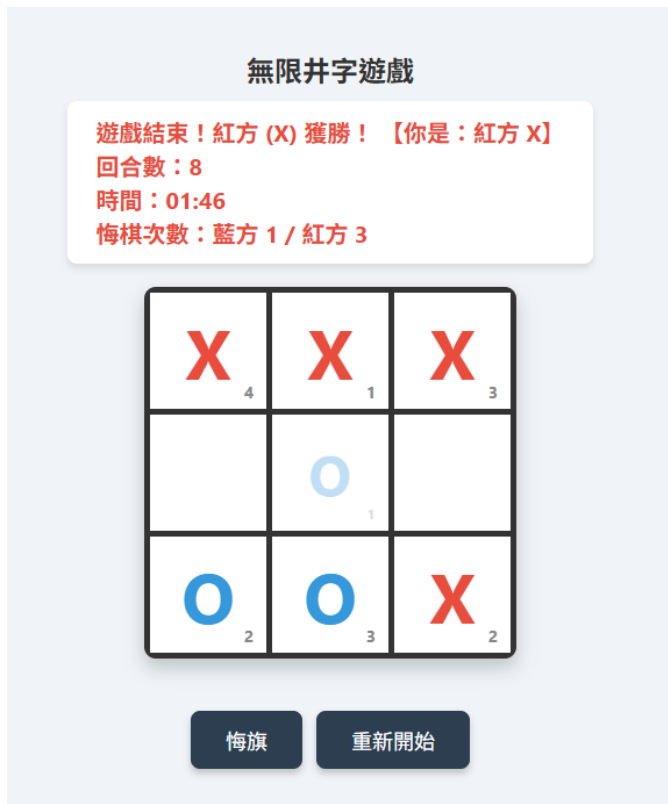


Fig. 1. 無限井字遊戲之對戰本體介面。為降低玩家的記憶負擔，系統於每個印記右下角附加了順序數字 (Badge)。數字為「1」的印記代表其為該玩家最早放置的符號，將在玩家進行下一步行動時優先被消除。

III. FIREBASE 雲端架構與即時同步機制

A. RTDB 與 WebSocket 事件驅動

傳統的 Web 多人遊戲常依賴 AJAX 短輪詢 (Short Polling) 來更新對戰狀態，這不僅大幅增加伺服器負載，更會導致畫面更新存在延遲。本專案核心採用 Firebase Realtime Database (RTDB) 構建數據中樞，其底層通訊基於 WebSocket 協定，建立了客戶端與伺服器間的持久性雙向連線。透過呼叫 `onValue()` 函數，前端轉變為「事件驅動 (Event-Driven)」模型：只要雲端 JSON 樹 (JSON Tree) 中的 `board` 或 `queues` 節點發生任何微小的寫入改變，Firebase 伺服器便會主動推播 (Push) 更新至所有連接的客戶端，達成真正的毫秒級狀態同步。

B. 輕量化 JSON 資料結構設計

為了使網路傳輸成本最小化，遊戲房間的對戰資料被精心展平為單一的 NoSQL JSON 節點。該節點的屬性涵蓋了：房間狀態 (status)、當前回合 (currentPlayer)、二維棋盤陣列 (board)、雙方玩家的動態 FIFO 佇列 (queues)，以及悔棋次數 (undoCounts)。客戶端僅需針對特定節點執行 update()，即可完成複雜的棋局變更。



Fig. 2. 遊戲大廳等待介面。此模組採用輕量級的 CSS Flexbox 佈局，並透過 JavaScript 定時器即時更新等待秒數。一旦後端偵測到挑戰者加入，頁面將立即解鎖並跳轉。

C. 原子化交易配對 (Atomic Transaction) 與併發控制

在大廳配對階段 (如圖 2 所示)，若有多名玩家在同一毫秒點擊加入，傳統的「讀取-修改-寫入」邏輯極易引發競爭危害 (Race Condition)，導致三人或以上擠入同一個僅限兩人的房間。為解決此併發存取問題，系統導入了 Firebase 的 runTransaction 機制。此原子化操作保證了狀態變更的不可分割性：

- 1) **房主初始化**：第 1 位玩家寫入初始數據，房間狀態設為 waiting 並記錄 hostId。
- 2) **挑戰者加入**：第 2 位玩家成功讀取到 waiting 狀態後，將其變更為 playing。若期間狀態已被他人搶先修改，交易將自動回滾 (Rollback)。
- 3) **自動跳轉**：雙方客戶端透過事件監聽偵測到 playing 後，即同步載入遊戲本體頁面。

D. 伺服器端生命週期管理與斷線清理

為了打造「終極無幽靈、人走房毀」的連線體驗，系統高度依賴 Firebase 伺服器端的 onDisconnect 觸發器。當房主在大廳等待時，系統會為其綁定斷線炸彈 (myDisconnectRef.remove())；若房主意外斷線或關閉網頁，Firebase 伺服器偵測到 Socket 斷開後，會由後端自動清除該房間節點，徹底避免無效「幽靈房間」的產生。值得注意的是，在雙方配對成功並準備跳轉至遊戲本體之前，客戶端必須精準呼叫 cancel() 解除此斷線鉤子，以防頁面重整的瞬間導致遊戲室遭到系統誤刪。

IV. 前端架構與技術統整

本專案將前端分為大廳配對與遊戲本體兩個獨立模組 (見表 I)。兩者之間透過 sessionStorage 進行輕量級的狀態傳遞 (如玩家角色 myRole)，實現了邏輯的解耦。

TABLE I
系統架構模組劃分

模組名稱	核心職責	關鍵技術
lobby.js	房間配對、並行控制	runTransaction
lobby.html	視覺等待提示、倒數計時	CSS Flexbox 佈局
script.js	FIFO 規則、獲勝判定	onValue 即時同步
main.html	棋盤渲染、回合與悔棋 UI	CSS Grid, DOM 操作

V. CONCLUSION

本專案成功開發並實作了無限井字遊戲，展現了結合前端動態渲染與後端即時資料庫的完整架構能力。透過 FIFO 佇列完美實現了遊戲規則的創新，並利用原子化交易與斷線偵測技術，建立了穩定且無衝突的即時配對系統。未來可進一步擴充排行榜系統或 Elo 積分演算法，以提升遊戲的競技性與社群互動性。